

ภาพรวมของบริการ (Service Overview)

AlertResponse Service

1. Service Owner

นายกิตติชัย เค้นสกุลประเสริฐ รหัสนักศึกษา 6609650079 ภาคพิเศษ

2. Service Purpose

AlertResponse Service เป็นบริการที่รับผิดชอบการสร้างและเผยแพร่การแจ้งเตือนภัยไปยังประชาชนในพื้นที่เป้าหมายผ่าน SMS และรวบรวมการตอบกลับของประชาชน (รับทราบ / ปลอดภัย / ต้องการความช่วยเหลือ) เพื่อสรุปสถานการณ์แบบ Real-time ช่วยให้หน่วยงานตัดสินใจจัดสรรความช่วยเหลือได้ตรงจุด

3. Pain Point ที่แก้ไข

เมื่อเกิดภัยพิบัติข้อมูลสถานการณ์มักกระจายหลายช่องทางและเปลี่ยนแปลงรวดเร็วทำให้ประชาชนสับสนและหน่วยงานไม่สามารถทราบได้อย่างชัดเจนว่าประชาชนในพื้นที่ได้รับข้อมูลแล้วหรือยังและมีใครบ้างที่ต้องการความช่วยเหลืออย่างเร่งด่วนในพื้นที่ภัยพิบัติ

3. Target Users

- ประชาชนในพื้นที่เสี่ยง: ผู้รับข้อมูลและแจ้งสถานะความปลอดภัย
- หน่วยงานภาครัฐ/ศูนย์ควบคุม: ผู้ติดตามภาพรวมและบริหารจัดการเหตุการณ์

4. Service Boundary

- In-scope Responsibilities (สิ่งที่บริการนี้รับผิดชอบ)
 - สร้างและจัดเก็บข้อมูลการแจ้งเตือน (Alert Master Data)
 - การจัดเก็บข้อมูลของเหตุการณ์ (Incident Type, Severity, Location) เพื่อบันทึกประวัติการแจ้งเตือน
 - การรวบรวมและตรวจสอบความถูกต้องของการตอบกลับจากประชาชน (Validation)
 - การคำนวณสถิติแบบ Atomic (Total Safe, Total Acknowledged, Total Need Help)
 - การส่งคำสั่งกระจายข้อความแจ้งเตือนแบบ Asynchronous ผ่านระบบคิว
- Out-of-scope / Not Responsible For (ไม่รับผิดชอบ)
 - การวิเคราะห์หรือตรวจจับการเกิดภัยพิบัติ การสั่งการหน่วยกู้ภัยหรือการจัดการ Logistics ของการช่วยเหลือ
 - การจัดการศูนย์พักพิงและทะเบียนประวัติผู้ลี้ภัย

5. Autonomy / Decision Logic

บริการมีความเป็นอิสระในการตัดสินใจเกี่ยวกับ:

- **การเปลี่ยนสถานะอัตโนมัติ:** เปลี่ยนสถานะเป็น ALERTED เมื่อเริ่มกระจายข้อความ และเปลี่ยนเป็น RESOLVED เมื่อมีการส่งจบภารกิจจากส่วนกลาง
- **การคัดกรองข้อมูล:** ปฏิเสธการบันทึกการตอบกลับสถานะหากการแจ้งเตือนนั้นถูกปิดไปแล้ว (Status = RESOLVED)
- **การจัดการความขัดแย้ง:**
รองรับ Idempotency โดยการปฏิเสธการนับคะแนนซ้ำจากเบอร์โทรศัพท์เดิมในเหตุการณ์เดิม

การตัดสินใจอิงจาก:

- citizen_response (1=acknowledged, 2=safe, 3=need_help)
- สถานะของ alert ปัจจุบัน
- จำนวน response ที่ได้รับ
- business rules ที่กำหนด

บริการสามารถตัดสินใจได้เองภายใต้ business rules ที่กำหนด โดยไม่ต้องรอ/ต้องรอการอนุมัติจากมนุษย์ในกรณีปกติ

- หาก response ไม่ตรง format → ปฏิเสธทันที
- หาก alert อยู่ในสถานะ RESOLVED → ไม่รับ response ใหม่
- หาก ส่งไม่สำเร็จ → retry อัตโนมัติไม่เกิน 3 ครั้ง

6. Owned Data

- **Alert Master Data:** ข้อมูลต้นแบบของการแจ้งเตือน รวมถึงข้อความประกาศ และค่า Snapshot ของพื้นที่เกิดเหตุ
- **Response Operational Data:** ข้อมูลรายละเอียดการตอบกลับรายบุคคล เช่น เบอร์โทรศัพท์ พิกัดที่ตอบกลับ และประเภทความต้องการความช่วยเหลือ
- **Summary Analytics State:** ตัวเลขสถิติรวมที่สรุปจากข้อมูลการตอบกลับ (Safe/Ack/Help)

7. Linked Data (Reference Only)

ก่อนเผยแพร่หรือปิดการแจ้งเตือน ระบบจะตรวจสอบสถานะของเหตุการณ์จาก Incident Service เพื่อให้มั่นใจว่า incident ยังอยู่ในสถานะที่เหมาะสม (เช่น ACTIVE)

- **Incident ID:** อ้างอิงรหัสเหตุการณ์จากระบบต้นทาง
- **Incident Status:** ตรวจสอบสถานะเหตุการณ์ปัจจุบัน (ACTIVE/RESOLVED) เพื่อควบคุมการแสดงผลบนหน้าเว็บของ Admin

8. Non-Functional Requirements

- **Reliability:** ระบบต้องรองรับการส่ง SMS ซ้ำ (Retry) เมื่อ Gateway มีปัญหา และต้องทำงานต่อเนื่องได้แม้ฐานข้อมูลบางส่วนล่าช้า
- **Data Integrity:** สถิติสรุปต้องมีความแม่นยำสูง (Atomic Update) ป้องกันปัญหาการนับพร้อมกัน (Race Condition)
- **Observability:** มีการบันทึก Logs ทุกขั้นตอน ตั้งแต่การสร้าง Alert จนถึงการตอบกลับครั้งสุดท้าย เพื่อให้สามารถตรวจสอบได้ง่าย

Synchronous Function Contract

Base URL: <https://1y2hd4pope.execute-api.us-east-1.amazonaws.com/v1>

API Contract #1: Create Alert

ข้อมูลทั่วไป

- **Name:** Create Alert
- **Method:** POST
- **Path:** /alerts
- **Type:** Synchronous

คำอธิบาย

ใช้สำหรับสร้างรายการแจ้งเตือนโดยอ้างอิงจากเหตุการณ์ (incident) ที่มีอยู่ใน Incident Service

1. ตรวจสอบความถูกต้องของข้อมูล
2. ตรวจสอบสถานะ incident จาก Incident Service (ต้องเป็น ACTIVE)
3. ดึงข้อมูล incident (incidentType, severity, location)
4. บันทึก snapshot ของ incident ภายใน AlertResponse Service
5. Publish AlertCreated event เพื่อให้กระบวนการส่ง SMS ทำงานแบบ asynchronous

Request

Path/Query Params

- ไม่มี

Headers

- Content-Type: application/json

- Authorization: Bearer <access_token>
- Idempotency-Key: <uuid>

Body:

```
{  
  
  "incidentId": "INC-12345",  
  
  "message": "เกิดเหตุน้ำท่วม โปรดอพยพไปยังจุดปลอดภัย"  
  
}
```

Validation Rules

- incidentId ต้องมีอยู่ใน Incident
- incident.status ต้องไม่เป็น RESOLVED
- message ต้องไม่ว่าง

Response

Success: 201 Created

```
{  
  
  "message": "สร้างการแจ้งเตือนสำเร็จ",  
  
  "alertId": "ALT-A1B2C3D4",  
  
  "incidentStatus": "ALERTED"  
  
}
```

Error:

- 400 Bad Request – ข้อมูลไม่ถูกต้องหรือไม่ครบถ้วน
- 401 Unauthorized – ไม่มีสิทธิ์ใช้งาน
- 404 Not Found – ไม่พบ incidentId
- 409 Conflict – มี alert ที่ active อยู่แล้ว
- 500 Internal Server Error – ระบบขัดข้อง

Dependency / Reliability

- พึงพาฐานข้อมูลหลัก (Database)
- Publish AlertCreated event ไปยัง Message Broker
- รองรับ idempotency เพื่อป้องกันการสร้าง Alert ซ้ำ
- ไม่รอการส่ง SMS (แยกเป็น asynchronous process)

API Contract #2: Record Citizen Response

ข้อมูลทั่วไป

- **Name:** Record Citizen Response
- **Method:** POST
- **Path:** /alerts/{alertId}/responses
- **Type:** Synchronous

คำอธิบาย

ใช้สำหรับบันทึกการตอบกลับของประชาชนผ่าน SMS Gateway โดยรองรับการตอบกลับแบบ
รองรับการตอบกลับประเภท:

- ACK (รับทราบ)
- SAFE (ปลอดภัย)
- HELP (ต้องการความช่วยเหลือ)

ระบบจะ validate และบันทึกข้อมูลลงฐานข้อมูลทันที

Request

Path/Query Param

- alertId (Path Parameter) – รหัสแจ้งเตือนที่ประชาชนตอบกลับ

Headers

- Content-Type: application/json
- Idempotency-Key: <uuid>

Body

```
{
  "phoneNumber": "0812345678",
  "replyText": "SAFE"
}
```

```
}
```

Validation

- alertId ต้องมีอยู่จริงในระบบ
- การแจ้งเตือนนั้นต้องไม่อยู่ในสถานะ RESOLVED
- หนึ่งเบอร์โทรศัพท์สามารถตอบกลับได้เพียงครั้งเดียวต่อหนึ่ง Alert (Idempotency)

Response

Success: 200 OK

```
{
```

```
  "message": "บันทึกสถานะสำเร็จ"
```

```
}
```

Error:

- 400 Bad Request – รูปแบบข้อมูลไม่ถูกต้อง
- 404 Not Found – ไม่พบ alertId
- 409 Conflict – มีการบันทึกซ้ำ
- 500 Internal Server Error – ระบบขัดข้อง

Dependency / Reliability

- อัปเดต summary counters แบบ atomic operation
- รองรับ retry จาก SMS Gateway กรณี network timeout
- รองรับ idempotency เพื่อป้องกันการนับซ้ำ

API Contract #3: Get Alert Summary

ข้อมูลทั่วไป

- **Name:** Get Alert Summary
- **Method:** GET
- **Path:** /alerts/latest
- **Type:** Synchronous
- **คำอธิบาย:**

- ใช้สำหรับดึงข้อมูลรายละเอียดของการแจ้งเตือน (Alert) รวมถึงข้อมูลสรุปสถิติ (Summary Counters) ของการตอบกลับจากประชาชน
- ข้อมูลที่ได้จะนำไปแสดงผลแบบ Real-time บน Dashboard ของ Operator เพื่อใช้ในการตัดสินใจ

Request

- **Path/Query Params:**
 - alertId (Path Parameter) – รหัสแจ้งเตือนที่ต้องการดูสรุปผล
- **Headers:**
 - Authorization: Bearer <access_token>
- **Body:**
 - ไม่มี

Validation Rules

- alertId ต้องมีรูปแบบที่ถูกต้องและมีข้อมูลอยู่ในระบบ (DynamoDB)

Response

Success: 200 OK

JSON

```
{
  "alertId": "ALT-A1B2C3D4",
  "incidentId": "INC-12345",
  "status": "ALERTED",
  "message": "ระวัง! น้ำป่าไหลหลาก...",
  "severitySnapshot": "HIGH",
  "totalSafe": 150,
  "totalAcknowledged": 85,
  "totalNeedHelp": 12,
  "updatedAt": "2026-05-17T10:00:00Z"
}
```

- **Error:**
 - 400 Bad Request – รูปแบบ alertId ไม่ถูกต้อง
 - 401 Unauthorized – ไม่มีสิทธิ์ใช้งาน
 - 404 Not Found – ไม่พบ alertId ในระบบ
 - 500 Internal Server Error – ระบบขัดข้อง ไม่สามารถเชื่อมต่อกับฐานข้อมูลได้

Dependency / Reliability

- ฟังก์ชันการอ่านข้อมูลจากฐานข้อมูลหลัก (DynamoDB)

- ดึงข้อมูล Summary Counters ที่ถูกอัปเดตแบบ Atomic Operation ทำให้ได้ตัวเลขที่เป็นปัจจุบันที่สุดเสมอ

Asynchronous Function Contract

Message Contract #1: AlertCreated

ข้อมูลทั่วไป

- **Message Name:** AlertCreated
- **Interaction Style:** Event-Driven (Publish/Subscribe)
- **Producer:** CreateAlertHandler
- **Consumer:** SMSDispatchWorker
- **Channel/Queue:** Amazon SQS (Queue)
- **Version:** v1.1 (Latest)

คำอธิบาย

เหตุการณ์นี้ถูก publish เมื่อ Alert ถูกสร้างเสร็จและอยู่ในสถานะ ACTIVED

SMS Dispatcher Service จะ subscribe event นี้ และดำเนินการ broadcast ข้อความแจ้งเตือนไปยังประชาชนในพื้นที่

ใช้ asynchronous messaging เพราะการ broadcast SMS เป็นงานที่ใช้เวลานานและไม่ควร block API หลัก การแยกเป็น event-driven ช่วยให้ระบบตอบกลับ client ได้รวดเร็วและเพิ่ม reliability เมื่อเกิด failure

Request

Message Headers

- eventType = AlertCreated
- source = CreateAlertHandler

Message Body

```
{
  "alertId": "ALT-A1B2C3D4",
  "incidentId": "INC-12345",
  "message": "ระวัง! น้ำป่าไหลหลาก...",
  "status": "ALERTED",
  "severity": "HIGH",
  "locationSnapshot": "พื้นที่เขตอำเภอเมือง จ.เชียงใหม่",
  "createdAt": "2026-05-17T10:00:00Z"
}
```

Field	Type	Required	Description
alertId	String	yes	รหัสอ้างอิงการแจ้งเตือน (เช่น ALT-xxxx)
incidentId	String	yes	รหัสเหตุการณ์ที่อ้างอิงจาก Incident Service
message	String	yes	ข้อความประกาศที่ส่งหาประชาชน
status	Enum	yes	CREATED / ALERTED / RESOLVED / FAILED

severitySnapshot	String	yes	ระดับความรุนแรง ณ เวลาที่สร้าง Alert
totalSafe	Number	yes	ยอดรวมประชาชนที่แจ้งว่าปลอดภัย
totalAck	Number	yes	ยอดรวมประชาชนที่แจ้งว่ารับทราบ
totalHelp	Number	yes	ยอดรวมประชาชนที่ขอความช่วยเหลือ
updatedAt	String	yes	เวลาที่มีการปรับปรุงข้อมูลล่าสุด

Field

Definition:

Validation Rules

1. alertId ต้องไม่ว่าง
2. severity {LOW, MODERATE, HIGH, CRITICAL}
3. message ต้องมีความยาวไม่เกิน 500 ตัวอักษร

Response

Message Headers

- messageId
- timestamp
- eventType = SMSDispatchResult
- source = SMSDispatcherService

Success Message Body

```
{  
  
  "alertId": "ALT-20260218-001",  
  "status": "SMS_DISPATCHED",  
  "processedAt": "2026-02-18T10:16:10Z",  
}
```

```
"failureReason": null
}
```

Reject/Error Message Body

```
{
  "alertId": "ALT-20260218-001",
  "status": "FAILED",
  "processedAt": "2026-02-18T10:16:10Z",
  "failureReason": "SMS Gateway Unavailable"
}
```

Field Definition:

Field	Type	Required	Description
alertId	String	yes	รหัสแจ้งเตือน
status	String	yes	สถานะการส่ง
processedAt	DateTime	yes	เวลาที่ประมวลผล
failureReason	String	No	เหตุผลหากล้มเหลว

Validation Rules

1. $\text{status} \in \{\text{SMS_DISPATCHED}, \text{FAILED}\}$
2. หาก $\text{status} = \text{FAILED} \rightarrow$ ต้องมี failureReason
3. ต้องมี retry อย่างน้อย 3 ครั้งก่อนส่งไป Dead Letter Queue (DLQ)

Message Contract #2: SMSDispatchResult (Subscriber)

ข้อมูลทั่วไป

- **Message Name:** SMSDispatchResult
- **Interaction Style:** Event-Driven (Publish/Subscribe)
- **Producer:** SMSDispatchWorker
- **Consumer:** AlertResponse Service
- **Channel/Topic:** Amazon SNS (หรือ SQS Callback)
- **Version:** v1.1 (Latest)
- **คำอธิบาย:**
 - เหตุการณ์นี้จะถูก Publish มาจาก Worker หลังจากกระบวนการกระจาย SMS ไปยัง Gateway เสร็จสิ้น (ไม่ว่าจะสำเร็จหรือล้มเหลว)
 - AlertResponseSubscriber จะคอยฟัง (Subscribe) Event นี้จาก SNS เพื่อนำผลลัพธ์มาอัปเดตฟิลด์ alert.status ในฐานข้อมูลหลัก (เปลี่ยนเป็น ACTIVE หรือ FAILED)
 - การทำงานส่วนนี้เป็น Asynchronous แบบ Fire-and-Forget ช่วยให้การเปลี่ยนสถานะทำได้โดยไม่กระทบการทำงานของระบบฝั่งหน้าบ้าน

Request (Message Received from SNS)

- **Message Headers:**
 - messageId (UUID)
 - eventType = SMSDispatchResult
 - source = SMSDispatcherService

Message Body:

JSON

```
{
  "alertId": "ALT-A1B2C3D4",
  "status": "SMS_DISPATCHED",
  "processedAt": "2026-05-17T10:05:00Z",
  "failureReason": null
}
```

●

Field Definition:

- **alertId** : รหัสแจ้งเตือนที่ต้องการอัปเดต
- **Status** : SMS_DISPATCHED (สำเร็จ) หรือ FAILED (ล้มเหลว)
- **processedAt** : เวลาที่ประมวลผลเสร็จสิ้น
- **failureReason** : ระบุสาเหตุหากส่งไม่สำเร็จ (เช่น Gateway Error)

Validation Rules

- alertId ต้องไม่ว่าง และต้องมีอยู่ในฐานข้อมูล
- status ต้องเป็นค่า SMS_DISPATCHED หรือ FAILED เท่านั้น
- หาก status = FAILED ควรมีการบันทึก failureReason ลง Logs เสมอ

Response / Action Taken (Internal)

- **Success:**
 - ระบบทำการอัปเดต alert.status ใน DynamoDB สำเร็จ
 - คืนค่า HTTP 200 กลับไปยัง SNS (เพื่อยืนยันการรับและประมวลผล Message สำเร็จ)
- **Error / Retry Policy:**
 - หากการอัปเดตฐานข้อมูลล้มเหลว (เช่น Database timeout) ระบบจะใช้ Retry Policy ของ SNS/SQS ในการส่ง Message มาให้ประมวลผลใหม่

- หาก Retry เกินจำนวนครั้งที่กำหนด Message จะถูกส่งไปยัง Dead Letter Queue (DLQ) เพื่อรอให้ทีม Ops ตรวจสอบ

Service Data

1) Alert Master Data (Owned by this service)

Field Name	Type	Required	Description	Example
alertId	string	Y (Primary Key)	รหัสแจ้งเตือน	ALT-A1B2C3D4
incidentId	string	Y (Reference Only)	อ้างอิง Incident จาก Incident Service	INC-12345
message	string	Y	ข้อความแจ้งเตือนที่ถูก generate แล้ว	น้ำท่วมระดับสูง โปรดอพยพ
severitySnapshot	enum	Y	เก็บ severity ตอนสร้าง (low/moderate/high/critical)	HIGH
locationSnapshot	Object (GeoJSON)	Y	พื้นที่ได้รับผลกระทบ ณ เวลาสร้าง Alert	{ "type": "Polygon", ... }
status	enum	Y	สถานะ: CREATED, ALERTED, RESOLVED, FAILED	ACTIVE
severitySnapshot	integer	Y	ระดับความรุนแรง ณ เวลาที่ประกาศ	1250
locationSnapshot	integer	Y	พื้นที่ที่ได้รับผลกระทบ (Snapshot)	800
totalSafe	integer	Y	ยอดรวมประชาชนที่แจ้งว่าปลอดภัย	350
totalAck	integer	Y	ยอดรวมประชาชนที่แจ้งว่ารับทราบ	100
totalNeedHelp	string	N	ยอดรวมประชาชนที่ขอความช่วยเหลือ	system
createdAt	datetime	Y	เวลาที่สร้างรายการ	2026-05-17T10:00 :00Z
updatedAt	datetime	Y	เวลาที่อัปเดตสถิติล่าสุด	2026-05-17T10:4 5:00Z

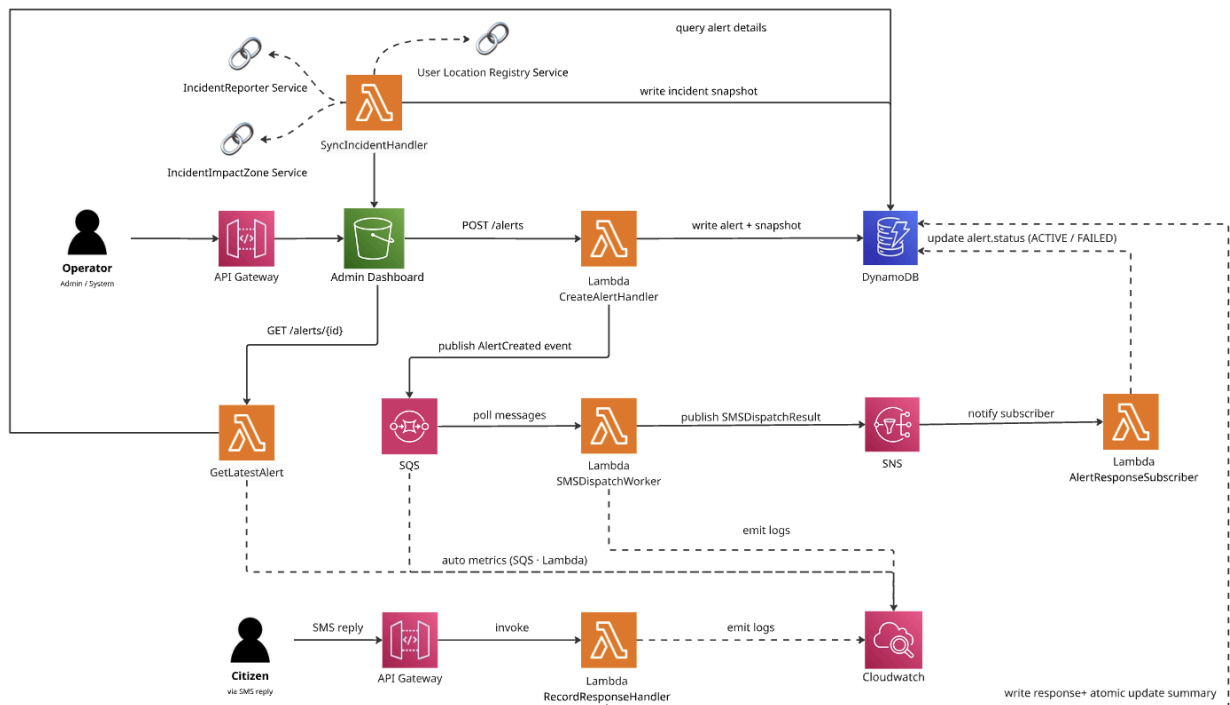
severitySnapshot เพราะ Incident severity อาจเปลี่ยนภายหลัง

ถ้าไม่ snapshot ประวัติ alert จะไม่ตรงกับเหตุการณ์ตอนส่งจริง

2) Citizen Response Data (Owned by this service)

Field Name	Type	Required	Description	Example
responseld	string	Y (Primary Key)	รหัสการตอบกลับ	RES-98765
alertId	String	Y (Foreign Key)	อ้างอิงรหัสการแจ้งเตือน	ALT-A1B2C3D4
phoneNumber	String	Y	เบอร์โทรศัพท์ผู้ตอบกลับ	0812345678
replyText	Enum	N	ประเภท: SAFE, ACKNOWLEDGE, NEED_HELP	SAFE
createdAt	DateTime	Y	เวลาที่ส่งคำตอบกลับ	026-05-17T10:15:00Z

Service Architecture



Components (องค์ประกอบของระบบ)

1. ผู้ใช้งาน (Users)

- **Operator (Admin/System):** เจ้าหน้าที่ผู้มีอำนาจตัดสินใจ ใช้เรียก API เพื่อสั่งการกระจายข่าวและติดตามยอดผู้ขอความช่วยเหลือแบบ Real-time
- **Citizen:** ประชาชนในพื้นที่เสี่ยงภัย รับข่าวสารผ่านอุปกรณ์พกพา และสามารถโต้ตอบสถานะความปลอดภัยกลับสู่ระบบได้ทันที

2. จุดรับคำขอ (API Gateway)

- ทำหน้าที่เป็นประตูหน้าบ้านรับคำขอแบบ Synchronous ทั้งหมด ได้แก่:
 - **POST /alerts :** รับคำสั่งสร้างการแจ้งเตือน
 - **GET /alerts/latest :** ดึงข้อมูลสรุปสถิติล่าสุด (ใช้ใน Dashboard)
 - **POST /alerts/{id}/responses :** รับข้อมูลการตอบกลับสถานะของประชาชน

3. กลุ่มประมวลผลแบบทันที (Synchronous Lambda Handlers)

- **CreateAlertHandler:** ตรวจสอบสิทธิ์และข้อมูลเหตุการณ์จากบริการภายนอก -> บันทึก Snapshot ลง DynamoDB -> ส่ง Message เข้า SQS -> ตอบกลับ 201 Created ทันที
- **GetAlertSummaryHandler:** ดึงข้อมูลสถิติล่าสุด (Status, Safe, Help) จาก DynamoDB มาแสดงผล
- **RecordResponseHandler:** รับคำตอบจากประชาชน -> ตรวจสอบ Validation -> อัปเดตยอด Counter ในตาราง **AlertData** แบบ Atomic Update และบันทึกประวัติลงตาราง **ResponseData**

4. บริการของเพื่อนในชั้นเรียน (Classmate Services - External)

- **IncidentReporter Service:** บริการต้นทางที่ระบบเราไปดึง **Incident Type** และ **Severity** มาใช้งาน
- **IncidentImpactZone Service:** บริการที่ให้ข้อมูลพิกัดหรือพื้นที่เป้าหมาย (Location) สำหรับการประกาศภัย
- **User Location Registry (Sync):** "รายชื่อเบอร์โทรศัพท์ของประชาชน" ที่ลงทะเบียนหรือปรากฏตัวอยู่ในพื้นที่นั้น เป็นกลุ่มเป้าหมายในการส่ง SMS

5. กลุ่มประมวลผลเบื้องหลัง (Asynchronous Messaging & Workers)

- **Amazon SQS (Queue):** คิวสำหรับรับงาน **AlertCreated** เพื่อประกันว่าคำสั่งส่งข้อความจะไม่สูญหายและไม่ทำให้ค้าง
- **SMSDispatchWorker (Lambda):** ทำหน้าที่ดึงงานจากคิวไปส่งยัง Gateway (พร้อมระบบ Retry 3 ครั้งหากล้มเหลว)
- **AlertResponseSubscriber (Lambda):** คอยอัปเดตสถานะสุดท้ายของ Alert ใน DynamoDB เป็น **ALERTED** หรือ **FAILED** หลังจากทราบผลการส่ง

6. ฐานข้อมูลและระบบมอนิเตอร์ (Storage & Monitoring)

- **Amazon DynamoDB:** ฐานข้อมูล NoSQL ที่แยกเก็บ 2 Table หลักคือ **AlertData** (Master) และ **ResponseData** (Operational)
- **Amazon CloudWatch:** ใช้จัดเก็บ Logs การทำงานทั้งหมดและตั้งค่า Alert หากระบบเกิดข้อผิดพลาด

Explanation (อธิบายการทำงานของระบบ)

สถาปัตยกรรมของ AlertResponse Service ถูกออกแบบมาเพื่อรองรับงานที่ต้องการความรวดเร็ว (Synchronous) และงานที่ต้องใช้เวลาประมวลผลนาน (Asynchronous) อย่างเป็นระบบครบ โดยมีเวิร์กโฟลว์การทำงานดังนี้:

1. การสร้างและกระจายการแจ้งเตือน (Create & Broadcast Flow) เมื่อ admin ส่งคำสั่งสร้างการแจ้งเตือนผ่าน API Gateway ระบบจะทำงานแบบประสานพลังกับบริการอื่นเพื่อให้ได้ข้อมูลที่สมบูรณ์ที่สุด:

- **Data Enrichment:** `CreateAlertHandler` จะวิ่งไปดึงข้อมูลความรุนแรงและประเภทภัยจาก `IncidentReporter Service` และดึงขอบเขตพื้นที่จาก `IncidentImpactZone Service` * **Target Identification:** ระบบจะส่งพิกัดพื้นที่ไปถาม `User Location Registry` เพื่อขอรายชื่อเบอร์โทรศัพท์ของประชาชนที่อยู่ในบริเวณนั้นทันที
- **Non-blocking Operation:** หลังจากบันทึกข้อมูล Snapshot ลง DynamoDB แล้ว ระบบจะโยนงานเข้าคิว `Amazon SQS` และตอบกลับ `201 Created` ให้เจ้าหน้าที่ทันทีเพื่อให้ Dashboard พร้อมทำงานต่อโดยไม่ต้องรอผลการส่งข้อความ
- **Background Processing:** `SMSDispatchWorker` จะรับช่วงต่อจาก SQS เพื่อส่งข้อความแจ้งเตือน และเมื่อเสร็จสิ้นจะส่งสัญญาณผ่าน SNS ให้ `AlertResponseSubscriber` มาอัปเดตสถานะของ Alert เป็น `ALERTED` เพื่อเริ่มแสดงผลบนหน้าเว็บของประชาชน

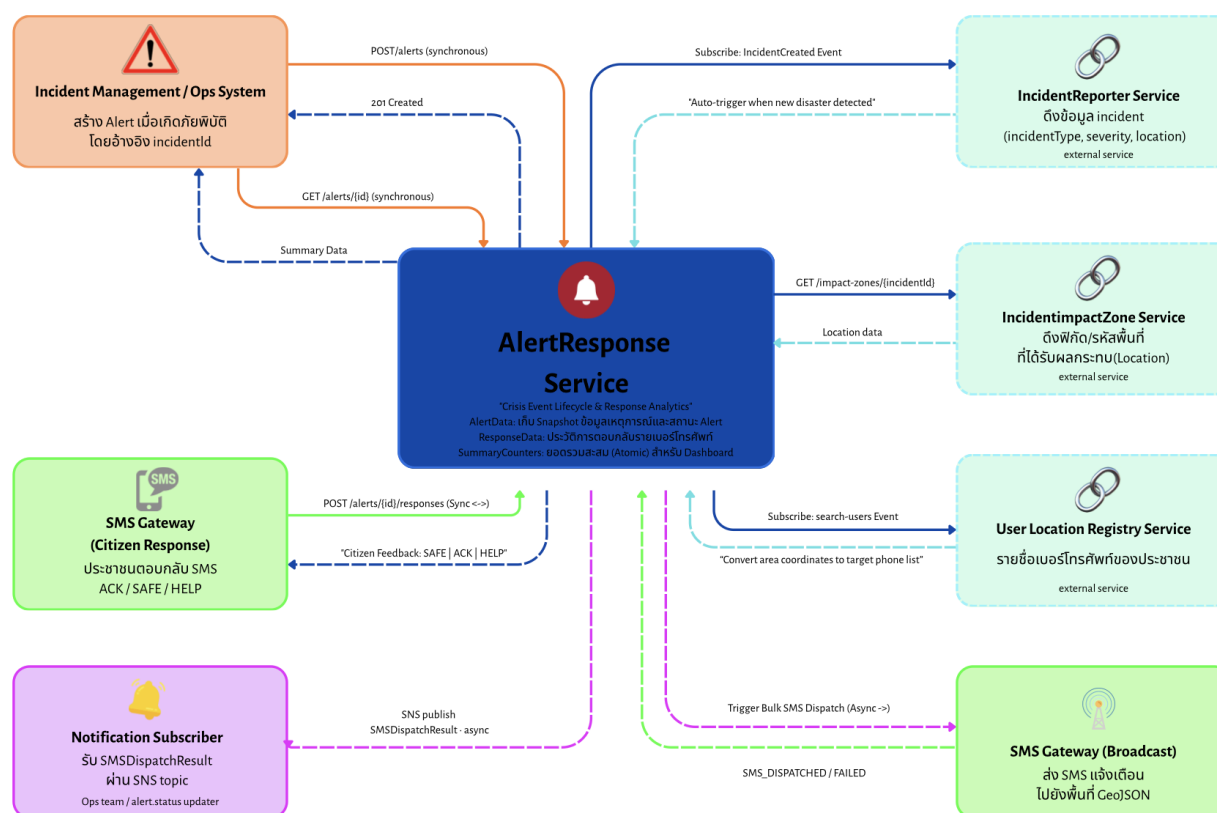
2. การโต้ตอบสถานะจากประชาชน (Citizen Response Flow) เมื่อประชาชนได้รับแจ้งเตือนและส่งสถานะตอบกลับ (เช่น `SAFE`, `ACKNOWLEDGE`, `NEED_HELP`) ผ่าน SMS:

- **Validation:** `RecordResponseHandler` จะตรวจสอบว่าการแจ้งเตือนนั้นยังไม่ถูกปิด (ยังไม่เป็น `RESOLVED`) และเบอร์โทรศัพท์นั้นยังไม่เคยตอบกลับมาก่อน (Idempotency)
- **Atomic Update:** ระบบจะทำการอัปเดตยอดสถิติรวม (Summary Counters) ใน DynamoDB แบบ **Atomic** ซึ่งเป็นการเพิ่มจำนวนนับในระดับฐานข้อมูลโดยตรง ช่วยป้องกันปัญหาข้อมูลคลาดเคลื่อนกรณีมีการตอบกลับเข้ามาพร้อมกันจำนวนมากในเสี้ยววินาที

3. การติดตามสถานการณ์และสรุปผล (Monitoring & Analytics Flow) ในระหว่างที่เหตุการณ์กำลังดำเนินอยู่ ระบบจะช่วยให้หน่วยงานรัฐเห็นภาพรวมสถานการณ์ได้แบบวินาทีต่อวินาที:

- **Real-time Dashboard:** เจ้าหน้าที่สามารถเรียกผ่าน `GetAlertSummaryHandler` เพื่อดึงตัวเลขล่าสุดของผู้ที่ปลอดภัยหรือต้องการความช่วยเหลือไปแสดงผล
- **System Health:** การทำงานทุกขั้นตอนจะถูกบันทึกลง `Amazon CloudWatch` ทั้งในรูปแบบของ Logs (เพื่อใช้ตรวจสอบย้อนหลัง) และ Metrics (เพื่อมอนิเตอร์ว่าระบบทำงานหนักเกินไปหรือมีการส่ง SMS ล้มเหลวหรือไม่) ทำให้มั่นใจได้ว่าระบบจะเสถียรแม้ในช่วงวิกฤตที่มีผู้ใช้งานหนาแน่น

Service Interaction



Service Interaction (การโต้ตอบระหว่างบริการ)

ระบบ **AlertResponse Service** ทำหน้าที่เป็นศูนย์กลางการสื่อสารในภาวะวิกฤต โดยมีการประสานงานกับบริการต่างๆ ทั้งในรูปแบบประสานเวลา (Synchronous) เพื่อความถูกต้องของข้อมูล และแบบไม่ประสานเวลา (Asynchronous) เพื่อประสิทธิภาพของระบบ ดังนี้:

Upstream Services เป็นส่วนที่ส่งข้อมูลเข้าสู่ระบบของเราเพื่อเริ่มการทำงานหรืออัปเดตสถานะ:

1. ระบบจัดการของเจ้าหน้าที่ (Operator / Admin Dashboard):
 - เรียกใช้ API แบบ Synchronous (POST /alerts) เพื่อสั่งสร้างการแจ้งเตือนด้วยมือในกรณีฉุกเฉิน
 - เรียกใช้ API แบบ Synchronous (GET /alerts/latest) เพื่อดึงข้อมูลสถิติแบบ Real-time ไปแสดงผลบน Dashboard
2. SMS Gateway (ช่องทางรับข้อมูลจากประชาชน):

- เมื่อประชาชนส่งสถานะกลับมา ระบบจะรับ Request แบบ **Synchronous (POST /responses)** เพื่อทำการตรวจสอบ (Validate) และอัปเดตยอด Counter ในฐานข้อมูลทันทีแบบ Atomic Update

3. Incident Service (Classmate Service - New Flow):

- ระบบของเราทำการ **Subscribe** รอรับ Event แบบ **Asynchronous** เมื่อมีการสร้างเหตุการณ์ใหม่ ทำให้ AlertResponse สามารถเริ่มกระบวนการเตรียมแจ้งเตือนได้โดยอัตโนมัติทันที

Downstream Services เป็นส่วนที่ไปดึงข้อมูลหรือส่งงานต่อเพื่อให้การแจ้งเตือนสมบูรณ์:

ส่วนนี้คือจุดที่ระบบของเราต้องไปดึงข้อมูลหรือโยนงานไปให้คนอื่นทำต่อ **ซึ่งครอบคลุมการเชื่อมต่อกับ Service ของเพื่อนร่วมชั้นเรียน (Classmate Services) ถึง 2 บริการครับ:**

1. IncidentReporter Service (Classmate Service):

- เรียกใช้งานแบบ **Synchronous** เพื่อดึงข้อมูล Snapshot ของเหตุการณ์ เช่น ประเภทภัย (Type) และความรุนแรง (Severity) มาใช้ประกอบการสร้างข้อความแจ้งเตือน

2. IncidentImpactZone Service (Classmate Service):

- เรียกใช้งานแบบ **Synchronous** เพื่อขอข้อมูลขอบเขตพื้นที่ที่ได้รับผลกระทบ (Location Geometry) ในรูปแบบ GeoJSON

3. User Location Registry Service (New Service):

- เรียกใช้งานแบบ **Synchronous** โดยการส่งข้อมูลพื้นที่จาก ImpactZone ไปเพื่อขอรับ **รายชื่อเบอร์โทรศัพท์ของประชาชน** ที่อยู่ในพื้นที่เป้าหมายกลับมาใช้ในการกระจาย SMS

4. SMS Gateway (Broadcast Flow):

- ส่งคำสั่งกระจายข้อความแบบ **Asynchronous** ผ่านระบบคิว (SQS) เพื่อส่งต่อให้ Worker ไปประมวลผลการส่งข้อความจำนวนมากเบื้องหลัง โดยไม่ทำให้ระบบหน้าบ้านต้องรอผล

Summary (สรุปภาพรวมการได้ตอบ)

- **Synchronous Interaction:** ใช้ในจุดที่ต้องการข้อมูลที่แม่นยำและเป็นปัจจุบันทันที (เช่น การดึงข้อมูลพื้นที่, การดึงเบอร์โทรศัพท์ และการรับสถานะช่วยเหลือจากประชาชน)
- **Asynchronous Interaction:** ใช้ในจุดที่การประมวลผลใช้เวลานานหรือต้องการลดการพึ่งพากันของระบบ (Decoupling) เช่น การรับแจ้งเหตุการณ์ใหม่จากเพื่อน และการกระจาย SMS จำนวนมาก เพื่อให้ระบบมีความทนทาน (Resiliency) และสามารถรองรับโหลตมหาศาลได้ในสถานการณ์ฉุกเฉิน

Dependency Mapping (แผนผังความพึ่งพา)

ในการออกแบบ Microservices การทำความเข้าใจว่าบริการของเราพึ่งพาใครและจะเกิด

อะไรขึ้นหากบริการเหล่านั้นขัดข้องเป็นสิ่งสำคัญมาก แผนผังนี้แสดงความสัมพันธ์ของ AlertResponse Service กับองค์ประกอบอื่นๆ:

ตารางวิเคราะห์ความพึ่งพา (Service Dependency Table)

Service	Type of Dependency	Interaction	Impact if Failure (หากระบบล่ม)
IncidentReporter	Hard Dependency	Asynchronous (Subscribe)	หากล่ม ระบบจะไม่สามารถดึงข้อมูล Snapshot (ประเภทภัย/ความรุนแรง) มาประกอบร่างข้อความแจ้งเตือนได้
IncidentImpact Zone (Classmate)	Hard Dependency	Synchronous (API Call)	หากล่ม ระบบจะไม่ทราบขอบเขตพื้นที่เป้าหมาย ทำให้กระบวนการหยุดชะงักทันที
User Location Registry	Hard Dependency	Asynchronous (Subscribe)	แม้จะรู้พื้นที่ แต่หากล่มจะไม่มีรายชื่อเบอร์โทรศัพท์ประชาชน ทำให้ไม่สามารถกระจาย SMS ได้
SMS Gateway (Broadcast)	Soft Dependency	Asynchronous (Queue)	หากล่ม งานจะยังคงค้างอยู่ใน SQS และระบบจะพยายาม Retry อัตโนมัติเมื่อ Gateway กลับมาใช้งานได้
Amazon DynamoDB	Critical Dependency	Internal Database	หากล่ม ระบบจะไม่สามารถบันทึกข้อมูล Alert หรือรับการตอบกลับจากประชาชนได้เลย

กลยุทธ์การจัดการเมื่อระบบพึ่งพาขัดข้อง (Resiliency Strategies)

- 1. **Decoupling via SQS:** การแยกส่วนกระจาย SMS ออกเป็น Asynchronous ช่วยให้หน้าบ้าน (Dashboard/API) ยังทำงานได้ปกติแม้ SMS Gateway จะล่าช้าหรือล่มชั่วคราว
- 2. **Data Snapshotting:** การเก็บ Snapshot จาก Incident และ ImpactZone มาไว้ในตาราง **AlertData** ของตนเอง ช่วยให้ระบบยังสามารถให้บริการข้อมูลแก่ประชาชนได้ แม้บริการต้นทางของเพื่อนจะขัดข้องในภายหลัง
- 3. **Idempotency Handling:** ระบบรองรับการส่งซ้ำจาก SNS/SQS หรือการกดซ้ำจากผู้ใช้งาน โดยใช้รหัสอ้างอิงเดิมเพื่อป้องกันการบันทึกข้อมูลซ้ำซ้อน
- 4. **Atomic Updates:** การอัปเดตยอดสรุปใน DynamoDB ใช้วิธี Atomic เพื่อรับประกันความถูกต้องของข้อมูล (Data Integrity) ในสภาวะที่มีโหลดสูง

ลิงค์ที่เกี่ยวข้อง

GitHub Repository : https://github.com/Kittithatdensakulprasert/Alertresponse_service

คลิปวิดีโอสาธิตการทำงานของระบบ :